

CSc 422, Program #2: Multithreaded Memory Allocator

Due date: Thursday, April 15th, at 8:00am. **No late assignments will be accepted.**

In this assignment you will implement a thread-safe `malloc` and `free`. This assignment must be done in C. You are required to do the following:

1. Develop (single-thread) versions of `malloc` and `free`. Their prototypes are:

```
void *myMalloc(int size);  
void myFree(void *ptr);
```

There also is an API function `myInit` that accepts input two parameters:

```
int myInit(int numThreads, int flag);
```

Any user program is *guaranteed* to call `myInit` before creating threads or invoking `myMalloc` or `myFree`. Function `myInit` returns zero on success (-1 on failure, which occurs if [system] `malloc` were to return an error). The first parameter is the number of threads that will be created (which for this part, is one), and the second is a flag (discussed later; whenever the number of threads is one, flag is irrelevant). User programs will never create more threads than cores, so this document uses the terms threads and cores interchangeably.

In `myInit`, the system `malloc` is used to allocate a total of 276,672 bytes, which is divided between metadata and data available to user programs (see below, as we are giving you code, and it correctly makes this division). **Please note that in this assignment, K refers to 1024, not 1000.** We will assume that the user never needs any more than 276,672 bytes; i.e., you will not need to increase the pool of memory that your `myMalloc` maintains to satisfy user requests. We will also assume that all `myMalloc` calls are either *small*, defined as less than or equal to 64 bytes, or *large*, defined as between 65 bytes and 1K, inclusive. There will never be a request larger than 1K, and you need not check requests for errors. You will always hand out either 64 or 1K byte chunks.

We have given you our implementation of `myMalloc` for small blocks only; you will need to extend this to handle both small and large blocks. You must use our circular, doubly-linked list format. Figure 1 shows a picture of our implementation of the small list. Our code initializes the pointers, as shown in the picture, via pointer arithmetic. When a block is returned to the user, we make sure the pointer returned is to the data and not to the size or the next or previous pointers. (Otherwise, the user will overwrite the metadata, there will likely be a future segmentation fault, and it will not be the fault of the user.)

As mentioned above, you are responsible for extending this to handle large blocks. **Note that you may not allocate more than 276,672 bytes of memory in your pool; 138,336 bytes for the small block list and 138,336 bytes for the large block list.** The `allocSize` field is

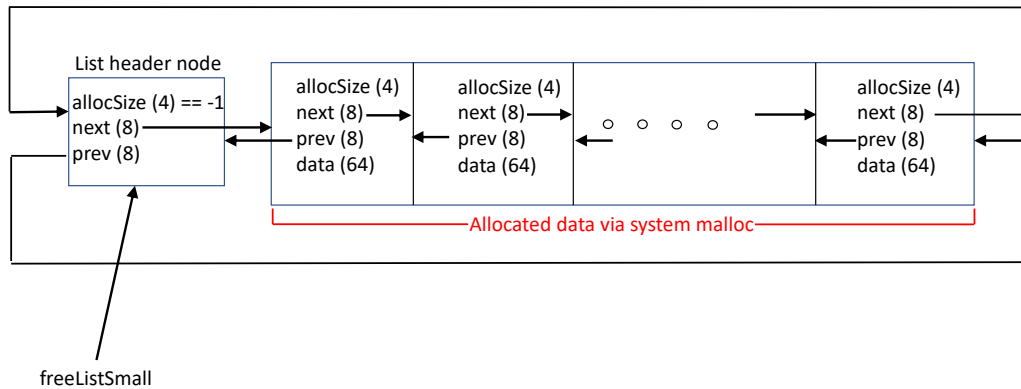


Figure 1: The small free list (bytes in parentheses).

used to hold the size of the block; this is needed in `myFree` in determining whether the block is returned to the small or large free list. Make sure that when you extend to large blocks, (system) `malloc` is only called once, and both the small and large lists are “embedded” in the allocated data. **You may not add any fields to the chunk data structure.**

2. Develop coarse-grain, concurrent versions of `myMalloc` and `myFree`. By coarse-grain I mean that you must use one lock for the entire two routines. You must use the locking routines available in `pthread` (`pthread_mutex_lock` and `pthread_mutex_unlock`); to initialize a lock, use `pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER`. A value of 1 for *flag* indicates the coarse-grain version. Note that user programs will contain `pthread`s for concurrency. The allocation rules are the same as above (pool of 276,672 bytes, split into 138,336 bytes for each of the small and large lists).
3. Develop fine-grain, concurrent versions of `myMalloc` and `myFree` in which you create a free list of memory **per core** (of size 276,672 bytes each; split into lists of 138,336 bytes as in part 1). You must support up to eight (8) concurrent threads. These per-thread lists will **not** be locked (each list will be accessed only by a unique thread). A value of 2 for *flag* indicates the fine-grain version. In this version, it is possible that a user program has at least one thread that allocates more than 276,672 bytes. Therefore, keep an **additional** shared list, that is thread-safe, also of size 276,672 bytes (which has the same rules as all other lists); this means you should have one overflow list of 64 byte blocks and one of 1K byte blocks. Cores use the overflow list if they run out of space on their local list. Assume that you never run out of space on the overflow list. Also assume that there will never be more than one thread per core. Note that as with the previous part, user programs will contain `pthread`s for concurrency.

4. While we are not *requiring* that you test your code, you would be well advised to do the following. First, write a single-threaded benchmark application that uses your `myMalloc` and `myFree` by repeatedly calling them (first `myMalloc` and then `myFree`) and accessing the data in between. This serves as your baseline. Second, write a multi-threaded benchmark application that uses your `myMalloc` and `myFree` by repeatedly calling them (first `myMalloc` and then `myFree`) and accessing the data (from different threads) in between. Third, make sure that the performance makes sense; in particular, the coarse-grain version should perform poorly under contention, while the fine-grain version should be contention-free unless many threads exhaust their local list.

Other requirements

- You are to write both the coarse- and fine-grain versions with one `myInit`, `myMalloc`, and `myFree`. This will require careful design. Consider that you can implement the coarse-grain version just like the fine-grain version if, upon initialization, you make the per-core lists all of size zero. (This will force use of the overflow list, which is equivalent to using just one list. As mentioned, make sure that you do not allocate more than 276,672 bytes for the overflow list.)
- You will need to be able to determine, in `myFree`, whether the pointer being returned is on the overflow list or not. To do this, you will need to store the start and end of the overflow list.
- Do *not* turn in any project that has debugging print statements (or any other kinds of print statements).
- Functions `myMalloc` and `myFree` **must be constant time operations**.
- You must create empty output files during execution as follows. The first time you allocate memory from list i (either a small or large allocation), create file `Id-i` (for example, if you are allocating from list 2, you create file `Id-2`). You must start i at 1 and increment by 1 each time memory is allocated from a new list. To do this, invoke the library call `system`, and use the `touch` command. My code does the following, assuming that `i` is the thread `id`, `str` is an array of at least 10 characters, and `seenId` is an integer array that is large enough to have an element for each core:

```
if (!seenId[id]) {
    // str must be a *local* variable to each thread!
    sprintf(str, "touch Id-%d\n", id);
    system(str);
    seenId[id] = 1;
}
```

The first time you allocate memory from the overflow list, create file `Overflow` in a similar way (it needs no id). The purpose of these file creations is so that I know (in a way that can be graded automatically) that you have allocated from the correct list. For the single-thread part, just touch `Id-1`. For the coarse-grain part, we will not look at (or grade you on) which files are touched. **Only touch each file once.**

- Because you will need to distinguish between threads, you will want to use following functions:

```
int pthread_key_create(pthread_key_t *key, void (*destructor)(void*));
int pthread_setspecific(pthread_key_t key, const void *value);
void *pthread_getspecific(pthread_key_t key);
```

The purpose of these functions is to determine which thread is invoking `myMalloc` or `myFree`. For more details, check the man page or do an Internet search. We also provide an example, `sample_pthread_setspecific.c`, that shows how these functions work. Make sure that you are handing out a unique id to each thread, starting at 1—you will need to use synchronization. Also, note that `pthread_getspecific` returns `NULL` to a thread that has never previously invoked `pthread_setspecific` to store any per-thread data. Finally, please understand that `pthread_setspecific` takes a **pointer** as its second argument. If you pass the same integer in (incrementing it each time), that is a logical error, and your program will crash. You need to make sure that you are passing a pointer to a different piece of allocated data each time you call `pthread_setspecific`.

You should make the following assumptions.

- You can assume all threads will only free memory that they have allocated. In particular, thread A will not free memory that thread B has allocated.
- All pointers are 8 bytes.
- As mentioned above (but repeated here for convenience), assume that you never run out of space on the overflow list, that there will never be more than one thread per core, and that user programs will contain pthreads for concurrency.
- Any call to `myMalloc` must be for a positive size (i.e., zero is not allowed).

Testing

To test your code, you need to implement a driver function, in which `main` should be implemented, and link the testcase to your code. As mentioned previously, `myInit` is called in `main` and before any call to `myMalloc` or `myFree`. We provide a sample driver function in `driver.c`.

When we test your programs, we will be using our own driver files. Therefore, you **must** conform to this interface or you will fail our test cases. Your routines must not produce output; we will use our own driver files.

Please download the tarball from D2L. This tarball contains:

- `sample_pthread_setspecific.c`
- `driver.c`
- `myMalloc.h`
- `myMalloc.c`
- `myMalloc-helper.h`
- `myMalloc-helper.c`, and
- `Makefile`.

You may not modify `myMalloc.h`.

Submission

All submissions should be to Gradescope and must include `myMalloc.c`, `myMalloc-helper.h`, `myMalloc-helper.c`, and `Makefile`.